# 1.5. Verificación de los mecanismos de integración de los lenguajes de marcas con los lenguajes de programación de clientes Web (Criterio 1.5)

- **Resultado de Aprendizaje Asociado: RA1** (*Selecciona las arquitecturas y tecnologías de programación sobre clientes Web, identificando y analizando las capacidades y características de cada una*).

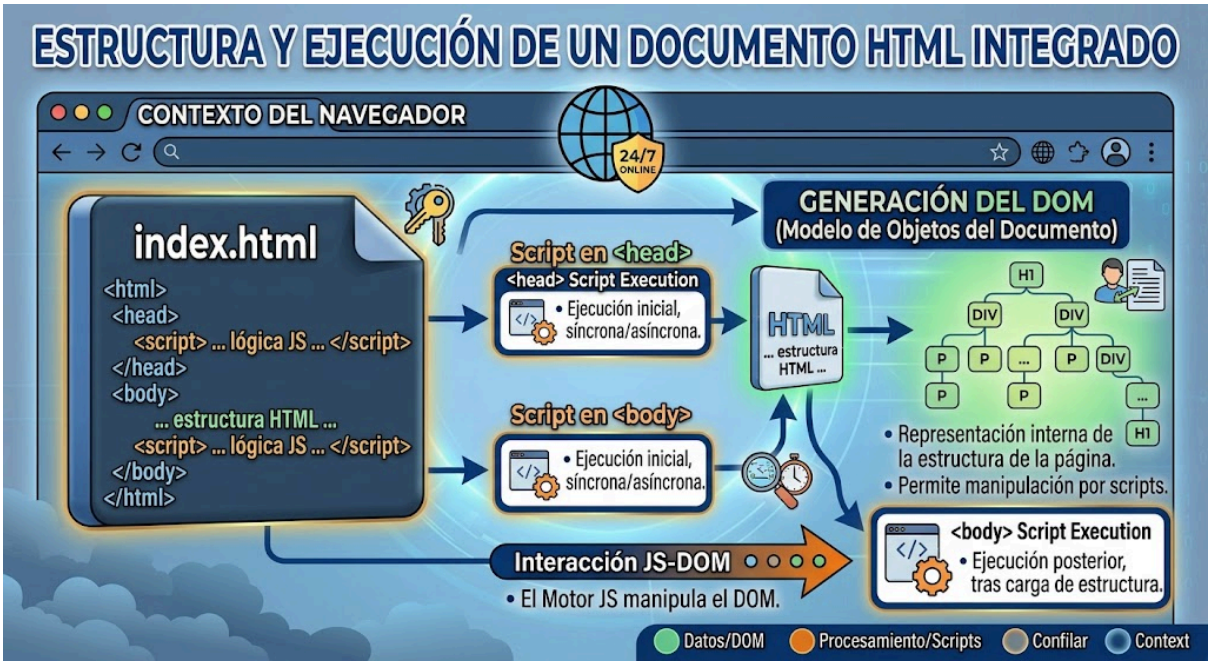
- **Criterio Curricular Oficial: CE 1.e** (Se han verificado los mecanismos de integración de los lenguajes de marcas con los lenguajes de programación de clientes web).

- **Ponderación del Criterio:** (**16,67% del RA1** | **0,833% sobre la calificación final del módulo**).

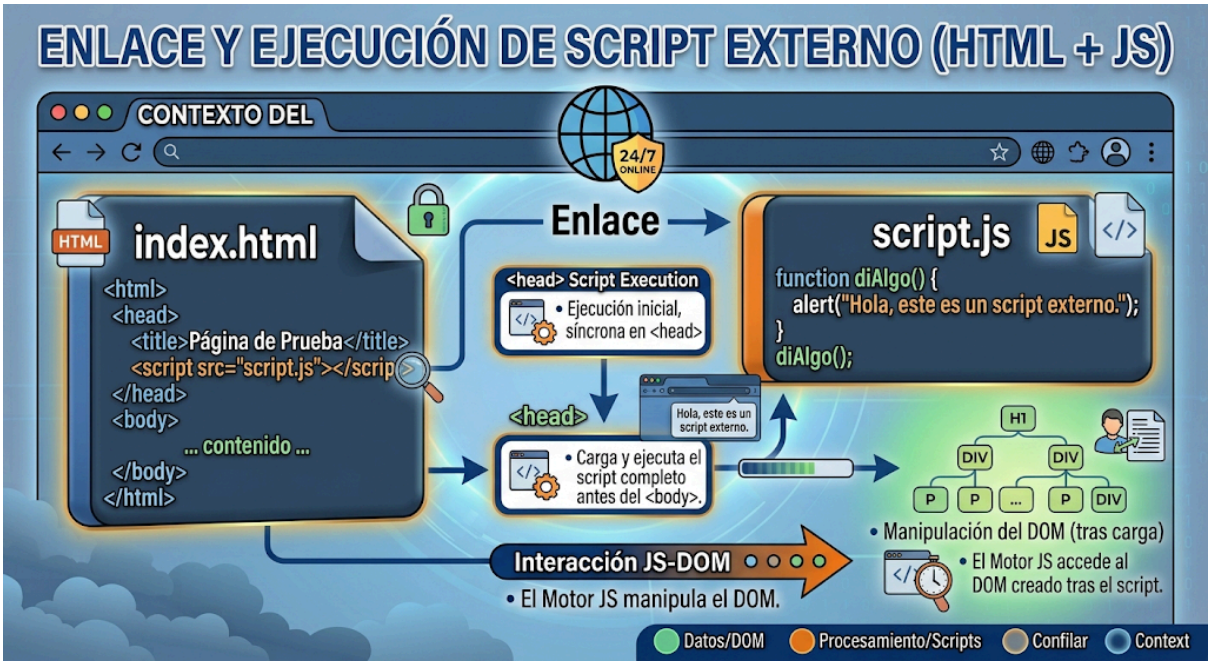
El lenguaje JavaScript no actúa de forma aislada en el navegador: se combina y complementa directamente con el código HTML de la página web. Para que el motor del navegador reconozca y ejecute las instrucciones de script, el estándar define mecanismos precisos de integración que determinan cómo, cuándo y en qué orden se procesa la lógica en relación con la estructura del documento.

OPCIONES DE INTEGRACIÓN DE JAVASCRIPT EN HTML

A) CÓDIGO EMBEBIDO (Todo en el mismo archivo)




B) FICHEROS SEPARADOS (Recomendado en proyectos profesionales)




## A. La Etiqueta <script> y su Evolución Técnica

La etiqueta estándar <script> es el contenedor oficial que el consorcio W3C define para insertar o enlazar código ejecutable dentro de un documento HTML:

- **Sintaxis actual (HTML5):** Basta con abrir y cerrar la etiqueta <script> y </script>. Los navegadores modernos asumen de forma predeterminada que el lenguaje interpretado es JavaScript.
- **Compatibilidad histórica (versiones legadas):** En versiones anteriores de JavaScript y HTML era común y obligatorio especificar el tipo MIME mediante el atributo type:

  <script type="text/javascript">

- Aunque los navegadores actuales aún lo reconocen por motivos de compatibilidad hacia atrás, ya no es necesario escribirlo.
- **La sintaxis estricta de cierre:** Una etiqueta <script> jamás puede cerrarse de forma abreviada (<script src="script.js" />). Debe incluir obligatoriamente su etiqueta de cierre correspondiente </script>, incluso cuando se enlazan ficheros externos vacíos de contenido interno.

## B. Código JavaScript en Ficheros Externos Separados

Consiste en mantener la estructura HTML en un archivo con extensión .html y extraer toda la lógica de programación a archivos de texto independientes con extensión .js.

### 1. Código del archivo index.html

```
<!DOCTYPE html>
<html>
<head>
```

```
  <title>Myfpschool</title>
  <!-- Enlace al fichero script.js ubicado en la misma carpeta -->
  <script src="script.js"></script>
</head>
<body>
</body>
</html>
```

**2. Código del archivo script.js**

```
// Definición de una función básica de saludo
function diAlgo()
{
  alert("hola"); // Muestra un cuadro modal emergente con el texto "hola"
}

// Invocación directa de la función al cargarse el fichero
diAlgo();
```

**3. Ventajas técnicas de utilizar ficheros externos**

- **Velocidad de carga y aprovechamiento de la memoria caché:** El navegador descarga el archivo .js una única vez y lo almacena en su memoria caché local. Si el usuario navega a otras páginas del mismo sitio web que usan ese script, no tiene que volver a descargarlo por la red, reduciendo el consumo de ancho de banda y acelerando la respuesta.
- **Independencia de facetas (Modularidad):** Se separa de forma estricta la estructura del contenido (HTML) del comportamiento dinámico (JavaScript), permitiendo que diseñadores y programadores trabajen simultáneamente sin pisarse el código.
- **Mantenimiento y reutilización de código:** Si se necesita corregir una función o actualizar un cálculo, se modifica un solo archivo .js y los cambios se reflejan inmediatamente en todas las páginas web que lo referencian.
- **Buenas prácticas de ordenación de carpetas:** En los proyectos profesionales, las empresas y los desarrolladores experimentados organizan los recursos situando los scripts dentro de un directorio dedicado denominado js o script (por ejemplo, <script src="./js/script.js"></script>).

## C. Código JavaScript Embebido dentro del HTML

Permite incrustar bloques de código JavaScript directamente entre las líneas de marcado del propio archivo HTML.

**Código de ejemplo (index.html)**

```
<!DOCTYPE html>
<html>
<head>
  <title>Myfpschool</title>
  <script>
    // Se declara la función dentro de la cabecera head
    function diAlgo()
```

```
  {
    alert("Hola");
  }
 </script>
</head>
<body>
 <!-- Se ejecuta la función en el cuerpo del documento body -->
 <script>
  diAlgo();
 </script>
</body>
</html>
```

**Ejemplo: Modificar el Contenido de la Página Web**

```
<!DOCTYPE html>
<html lang="es">
<head>
 <meta charset="UTF-8">
 <title>Modificando HTML con addEventListener</title>
</head>
<body>
 <h1>Modificando el código HTML</h1>
 <p id="prueba">Modificando el contenido.</p>

 <button type="button" id="btnCambiar">
  ¡Dale!
 </button>

 <script>
  function cambiarTexto() {
   // Usar textContent si solo cambias texto (más rápido y seguro que innerHTML)
   document.getElementById('prueba').textContent = 'CAMBIANDO el contenido!';
  }

  const boton = document.getElementById('btnCambiar');
  boton.addEventListener('click', cambiarTexto);
 </script>
</body>
</html>
```

**Características y desventajas de este enfoque**

- **Mismo resultado visual:** Tanto el enfoque embebido como el externo provocan exactamente el mismo efecto ante el usuario (ambos despliegan una alerta emergente con el saludo).

- **Dificultad de mantenimiento:** Diseminar bloques <script> desordenados a lo largo del <head> y del <body> convierte el código en un bloque difícil de depurar, entender y mantener a largo plazo.
- **Criterio de uso excepcional:** Solo se justifica el uso de JavaScript embebido cuando las líneas de código son mínimas, específicas para una sola página y no vayan a modificarse prácticamente nunca.

## D. Reglas de Ubicación: ¿Dentro de <head> o dentro de <body>?

El código JavaScript puede situarse indistintamente en la cabecera <head> o dentro del cuerpo <body> del documento. Sin embargo, la posición influye de manera determinante en cómo se procesa la página web:

- **Ubicación en el <head>:**
  - El navegador lee el documento de arriba a abajo. Si encuentra una etiqueta <script> en el <head>, detiene el análisis del HTML hasta que el script se descarga y se ejecuta por completo.
  - *Problema común:* Si ese script intenta acceder a un elemento del <body> (por ejemplo, mediante document.getElementById('prueba')), fallará con un error porque ese elemento aún no ha sido leído ni construido en el DOM.
- **Ubicación al final del <body> (Antes de </body>):**
  - Se considera la recomendación tradicional más eficaz: garantiza que todo el marcado HTML, textos e imágenes ya se han analizado e insertado en el DOM antes de que comience a ejecutarse la lógica de interacción.
- **Profundización moderna: Atributos defer y async (HTML5):**
  - Para scripts externos colocados en el <head>, los estándares modernos permiten evitar el bloqueo del navegador mediante dos atributos:
    - defer: Descarga el archivo JavaScript en segundo plano mientras el navegador sigue construyendo el HTML, pero retrasa su ejecución exacta hasta que el documento HTML se ha parseado por completo.

      <script defer src="script.js"></script>

    - async: Descarga el archivo en segundo plano y lo ejecuta de inmediato en cuanto termina la descarga, sin importar si el HTML ha terminado de leerse (útil para herramientas externas de analítica o contadores).

      <script async src="script.js"></script>

## E. Actividades Prácticas del Libro y de Consolidación

- **Actividad 1:** Crea un archivo llamado index.html y otro llamado script.js en la misma carpeta utilizando el código de ficheros separados. Ábrelo con un navegador web y comprueba que se muestra el cuadro de alerta con el texto "hola" al cargarse la página.

- **Actividad 2:** Crea el archivo index.html con el código embebido del apartado 1.5.B. Observa cómo se interrumpe la carga para mostrar el mensaje al navegante.
- **Práctica de Laboratorio Guiada (Reorganización de Proyecto):**
    1. Diseña una estructura de carpetas profesional:

```
mi_proyecto/
├── css/
│   └── estilos.css
├── js/
│   └── logica.js
└── index.html
```

    2. Traslada un bloque de script embebido que cambie el color de un botón a la carpeta js/logica.js.
    3. Enlaza el archivo desde el <head> de index.html utilizando la ruta relativa correcta (./js/logica.js) y comprueba en la pestaña *Network* (Red) de las herramientas del desarrollador (F12) que el fichero .js devuelve un código de estado 200 OK.

- **Realiza los ejercicios del apartado 1.2 y 1.4 haciendo uso de la estructura anterior.**